A Practical Guide to Reinforcement learning human feedback

TOPIC -- REINFORCEMENT LEARNING HUMAN FEEDBACK

-- 01 --

Direct preference optimization Direct preference optimization (DPO) is a technique to learn human preferences. Like RLHF, it has been applied to align pre-trained large language models using human-generated preference data. Unlike RLHF, however, which first trains a separate intermediate model to understand what good outcomes look like and then teaches the main model how to achieve those outcomes, DPO simplifies the process by directly adjusting the main model according to people's preferences. It uses a change of variables to define the "preference loss" directly as a function of the policy and uses this loss to fine-tune the model, helping it understand and prioritize human preferences without needing a separate step. Essentially, this approach directly shapes the model's decisions based on positive or negative human feedback. Recall, the pipeline of RLHF is as follows:

-- 02 --

Direct alignment algorithms Direct alignment algorithms (DAA) have been proposed as a new class of algorithms that seek to directly optimize large language models (LLMs) on human feedback data in a supervised manner instead of the traditional policy-gradient methods. These algorithms aim to align models with human intent more transparently by removing the intermediate step of training a separate reward model. Instead of first predicting human preferences and then optimizing against those predictions, direct alignment methods train models end-to-end on human-labeled or curated outputs. This reduces potential misalignment risks introduced by proxy objectives or reward hacking. By directly optimizing for the behavior preferred by humans, these approaches often enable tighter alignment with human values, improved interpretability, and simpler training pipelines compared to RLHF.

-- 03 --

In machine learning, reinforcement learning from human feedback (RLHF) is a technique to align an intelligent agent with human preferences. It involves training a reward model to represent preferences, which can then be used to train other models through reinforcement learning. In classical reinforcement learning, an intelligent agent's goal is to learn a function that guides its behavior, called a policy. The function is iteratively optimized to increase the reward signal

derived from the agent's task performance. However, explicitly defining a reward function that accurately approximates human preferences is challenging. Therefore, RLHF seeks to train a "reward model" directly from human feedback. The reward model is first trained in a supervised manner to predict if a response to a given prompt is good (high reward) or bad (low reward) based on ranking data collected from human annotators. This model then serves as a reward function to improve an agent's policy through an optimization algorithm like proximal policy optimization. RLHF has applications in various domains in machine learning, including natural language processing tasks such as text summarization and conversational agents, computer vision tasks like text-to-image models, and the development of video game bots. While RLHF is an effective method of training models to act better in accordance with human preferences, it also faces challenges due to the way the human preference data is collected. Though RLHF does not require massive amounts of data to improve performance, sourcing high-quality preference data is still an expensive process. Furthermore, if the data is not carefully collected from a representative sample, the resulting model may exhibit unwanted biases.

-- 04 --

Initialize the policy π φ R L {\displaystyle \pi _{\phi }^{RL}} to π S F T {\displaystyle \pi ^{SFT}} , the policy output from SFT. Loop for many steps. Initialize a new empty dataset D π φ R L {\displaystyle D_{\pi _{\phi }^{RL}}} . Loop for many steps Sample a random prompt x {\displaystyle x} from D R L {\displaystyle D_{RL}} . Generate a response y {\displaystyle y} from the policy π φ R L {\displaystyle \pi _{\phi }^{RL}} . Calculate the reward signal r θ (x , y) {\displaystyle r_{\theta }(x,y)} from the reward model r θ {\displaystyle r_{\theta }} . Add the triple (x , y , r θ (x , y)) {\displaystyle (x,y,r_{\theta }(x,y))} to D π φ R L {\displaystyle D_{\pi _{\phi }^{RL}}} . Update φ {\displaystyle \phi } by a policy gradient method to increase the objective function objective (φ) = E (x , y) ~ D π φ RL [r θ (x , y) − β log ■ (π φ RL (y | x) π SFT (y | x))] {\displaystyle {\text{objective}}(\phi)=E_{(x,y)\sim D_{\pi _{\phi }^{\text{RL}}}}\left[r_{\theta }(x,y)-\beta \log \left({\frac {\pi _{\phi }^{\text{RL}}(y|x)}{\pi ^{\text{SFT}}(y|x)}}\right)\right]}

-- 05 --

where K {\displaystyle K} is the number of responses the labelers ranked, r θ (x , y) {\displaystyle r_{\theta }(x,y)} is the output of the reward model for prompt x {\displaystyle x} and completion y {\displaystyle y} , y w {\displaystyle y_{w}} is the preferred completion over y l {\displaystyle y_{l}} , σ (x) {\displaystyle \sigma (x)} denotes the sigmoid function, and E [X] {\displaystyle E[X]} denotes the

expected value. This can be thought of as a form of logistic regression, where the model predicts the probability that a response y w {\displaystyle y_{w}} is preferred over y l {\displaystyle y_{l}} . This loss function essentially measures the difference between the reward model's predictions and the decisions made by humans. The goal is to make the model's guesses as close as possible to the humans' preferences by minimizing the difference measured by this equation. In the case of only pairwise comparisons, K = 2 {\displaystyle K=2} , so the factor of 1 / (K 2) = 1 {\displaystyle 1/{\tbinom {K}{2}}=1} . In general, all (K 2) {\displaystyle {\tbinom {K}{2}}} comparisons from each prompt are used for training as a single batch. After training, the outputs of the model are normalized such that the reference completions have a mean score of 0. That is, ∑ y r θ (x , y) = 0 {\textstyle \sum _{y}r_{\theta }(x,y)=0} for each query and reference pair (x , y) {\displaystyle (x,y)} by calculating the mean reward across the training dataset and setting it as the bias in the reward head.

Collecting human feedback Human feedback is commonly collected by prompting humans to rank instances of the agent's behavior. These rankings can then be used to score outputs, for example, using the Elo rating system, which is an algorithm for calculating the relative skill levels of players in a game based only on the outcome of each game. While ranking outputs is the most widely adopted form of feedback, recent research has explored other forms, such as numerical feedback, natural language feedback, and prompting for direct edits to the model's output. One initial motivation of RLHF was that it requires relatively small amounts of comparison data to be effective. It has been shown that a small amount of data can lead to comparable results to a larger amount. In addition, increasing the amount of data tends to be less effective than proportionally increasing the size of the reward model. Nevertheless, a larger and more diverse amount of data can be crucial for tasks where it is important to avoid bias from a partially representative group of annotators. When learning from human feedback through pairwise comparison under the Bradley–Terry–Luce model (or the Plackett–Luce model for K-wise comparisons over more than two comparisons), the maximum likelihood estimator (MLE) for linear reward functions has been shown to converge if the comparison data is generated under a well-specified linear model. This implies that, under certain conditions, if a model is trained to decide which choices people would prefer between pairs (or groups) of choices, it will necessarily improve at predicting future preferences. This improvement is expected as long as the comparisons it learns from are based on a consistent and simple rule. Both offline data collection models, where the model is learning by interacting with a static dataset and updating its policy in batches, as well as online data collection models, where the model directly

interacts with the dynamic environment and updates its policy immediately, have been mathematically studied proving sample complexity bounds for RLHF under different feedback models. In the offline data collection model, when the objective is policy training, a pessimistic MLE that incorporates a lower confidence bound as the reward estimate is most effective. Moreover, when applicable, it has been shown that considering K-wise comparisons directly is asymptotically more efficient than converting them into pairwise comparisons for prediction purposes. In the online scenario, when human feedback is collected through pairwise comparisons under the Bradley–Terry–Luce model and the objective is to minimize the algorithm's regret (the difference in performance compared to an optimal agent), it has been shown that an optimistic MLE that incorporates an upper confidence bound as the reward estimate can be used to design sample efficient algorithms (meaning that they require relatively little training data). A key challenge in RLHF when learning from pairwise (or dueling) comparisons is associated with the non-Markovian nature of its optimal policies. Unlike simpler scenarios where the optimal strategy does not require memory of past actions, in RLHF, the best course of action often depends on previous events and decisions, making the strategy inherently memory-dependent.